

Le (ou les ?) serveur(s)

Un serveur est un ordinateur ou un programme qui attend des requêtes et fournit un service en réponse.

Il faut voir ça comme un restaurant : un client commande un plat, le serveur va en cuisine pour demander aux cuisiniers de le préparer, puis le retourne au client une fois le plat terminé.

Exemples :

- Le navigateur demande une page → le serveur web l'envoie.
- Une application demande des données → le serveur applicatif exposant une API répond.
- Un utilisateur envoie un email → le serveur mail le transmet.
- Une application demande des données SQL → le serveur de base de données répond.

Le principe est toujours :

Client → Requête → Serveur → Réponse

Les 2 niveaux à distinguer

Le serveur physique (la machine)

C'est l'ordinateur.

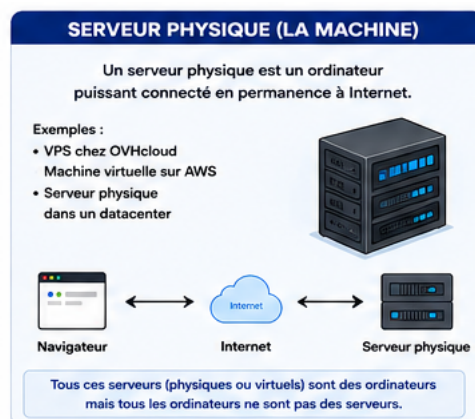


Figure 1: schéma d'un serveur physique

Les serveurs physiques sont la base matérielle sur laquelle tout le reste fonctionne ; il fournit : - du CPU (la puissance de calcul) - de la RAM (la mémoire) - du stockage (SSD, disques) - une connexion réseau

Dans le cloud, si on loue une machine virtuelle ou un serveur virtuel, on utilise une partie d'un serveur physique ; une VM / un VPS, c'est un ordinateur virtuel / serveur virtuel créé à l'intérieur d'un serveur physique.

Exemples :

- un VPS (serveur privé virtuel) chez OVHcloud
- une machine virtuelle sur AWS
- un serveur physique dans un datacenter

Navigateur → Internet → Serveur physique (OVH ou AWS par exemple)

Tous ces serveurs (physiques ou virtuels) sont des ordinateurs mais tous les ordinateurs ne sont pas des serveurs

Les logiciels serveurs installés dessus

Sur ces machines, on installe plusieurs programmes.

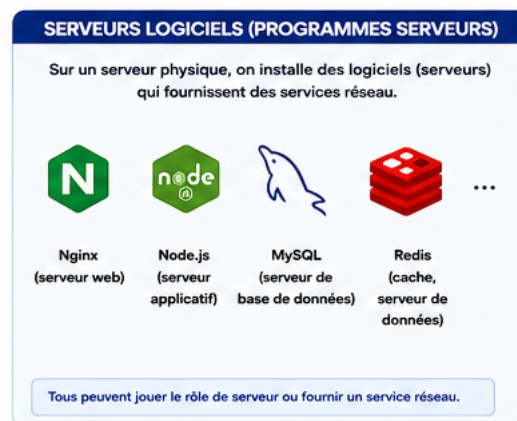


Figure 2: schéma d'un serveur logiciel

Par exemple :

Dans la machine Linux :

- Nginx
- Node.js
- MySQL
- Redis

Tous peuvent jouer le rôle de serveur ou fournir un service réseau.

Les principaux types de logiciels serveurs :

Chaque logiciel fournit un service différent.

C'est là qu'apparaissent les différentes catégories de serveur, notamment trouvées sur Internet.

Serveur Web

C'est le premier point d'entrée d'un site web. Il sert les fichiers statiques et peut transmettre certaines requêtes au backend.

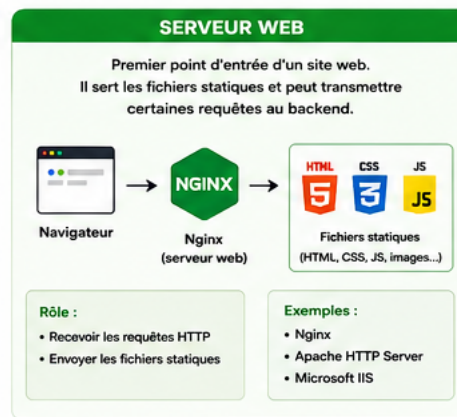


Figure 3: schéma d'un serveur web

Ce n'est pas forcément lui qui "héberge" les fichiers : il peut les servir depuis un disque local, ou les récupérer depuis un stockage (Amazon S3 au sein de AWS par exemple).

Rôle :

- recevoir les requêtes HTTP
- envoyer HTML, CSS, JS, images

Exemples :

- Nginx
- Apache HTTP Server
- Microsoft IIS

Navigateur → Nginx → index.html

Serveur Applicatif (backend)

Il traite les demandes de l'application, applique les règles métier ("un utilisateur doit être connecté pour publier", "un virement supérieur à 1000 € doit être validé", etc.) et génère les réponses.

Rôle :

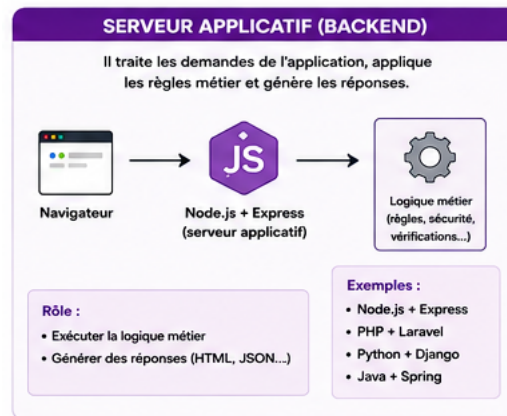


Figure 4: schéma d'un serveur backend

- exécuter la logique métier : c'est généralement le **backend**

Exemples :

- Node.js + Express
- PHP + Laravel
- Python + Django
- Java + Spring

“Créer un compte” → Serveur applicatif → Vérifie les données et crée l'utilisateur

Serveur de Base de Données

Il conserve les informations de l'application et permet de les lire, modifier ou supprimer.

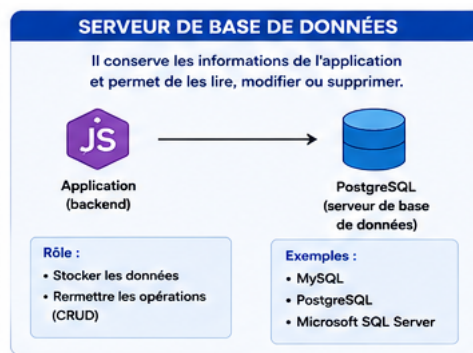


Figure 5: schéma d'un serveur de base de données

Rôle :

- stocker les données

Exemples :

- MySQL
- PostgreSQL
- Microsoft SQL Server

Node.js → PostgreSQL → Utilisateurs, Produits, Commandes

Serveur DNS

Il agit comme un annuaire d'Internet en traduisant les noms de domaine en adresses IP.

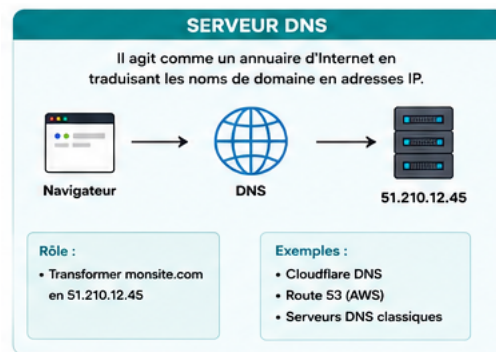


Figure 6: schéma d'un serveur dns

Rôle :

- Transformer monsite.com en 51.210.12.45

Sans DNS :

`https://51.210.12.45`

Avec DNS :

`https://monsite.com`

Serveur Mail

Il gère la circulation des emails entre les expéditeurs et les destinataires.

Rôle :

- envoyer
- recevoir

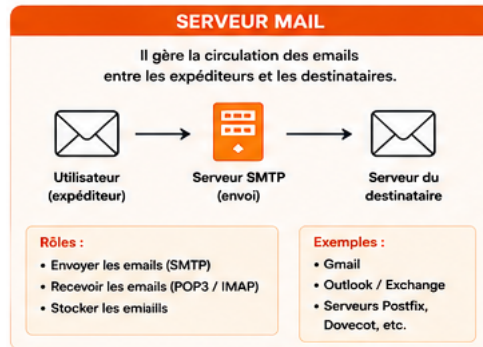


Figure 7: schéma d'un serveur mail

- stocker les emails

Exemple :

Gmail → Serveur SMTP → Serveur du destinataire

SMTP est l'abréviation de Simple Mail Transfer Protocol : protocole de communication utilisé pour envoyer et recevoir des messages électroniques sur Internet.

Serveur Proxy / Reverse Proxy

C'est l'intermédiaire entre le client et un autre serveur. Un reverse proxy protège et distribue les requêtes vers les services internes.

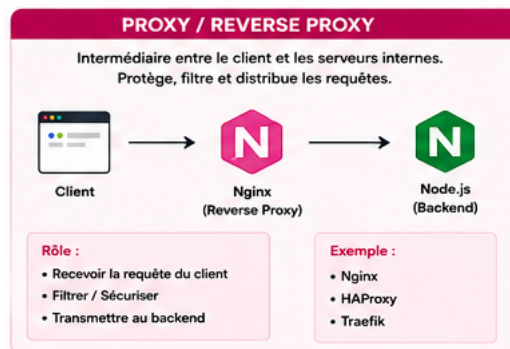


Figure 8: schéma d'un serveur proxy et reverse-proxy

Client → Nginx → Node.js

Le client ne communique jamais directement avec Node.js.

Reverse proxy :

- reçoit la requête
- filtre
- sécurise
- transmet au backend

Nginx : serveur web ou reverse proxy ?

Les deux : Nginx est un logiciel capable d'assurer plusieurs rôles :

- serveur web : il sert directement des fichiers.
- reverse proxy : Nginx reçoit la requête et la transmet à Node.js par exemple ; c'est un intermédiaire qui protège ou représente le serveur.
- les deux à la fois : c'est le cas le plus fréquent aujourd'hui.

Cas particuliers

Et Docker ?

Docker n'est PAS un type de serveur.

C'est un outil permettant d'exécuter des applications et services (donc des serveurs !) dans des conteneurs isolés.

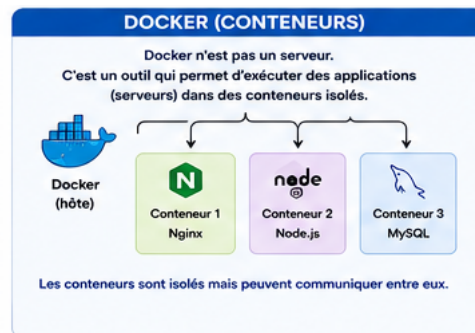


Figure 9: schéma d'un docker

Exemple :

Dans le docker :

- Conteneur 1 :
 - Nginx
- Conteneur 2 :
 - Node.js
- Conteneur 3 :
 - MySQL

Docker permet de faire tourner ces conteneurs isolément tout en les faisant communiquer.

Et le CDN ?

Un CDN est plutôt un réseau mondial de serveurs qui met en cache ou réplique du contenu (images, CSS, JS, vidéos, pages HTML, etc.) afin de les distribuer plus rapidement aux utilisateurs, en les servant sur plusieurs serveurs répartis dans le monde (pour chaque utilisateur, depuis le serveur le plus proche).

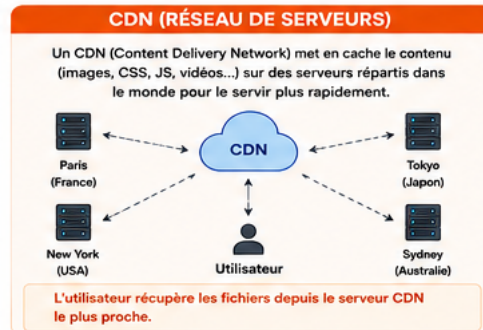


Figure 10: schéma d'un cdn

Mettre en cache, c'est faire une copie temporaire d'une donnée stockée près de l'utilisateur pour éviter de la redemander à l'origine.

Par exemple :

- Cloudflare
- Amazon CloudFront
- Google Cloud CDN

Ces entreprises possèdent des milliers de serveurs répartis dans le monde.

Exemple d'un site web hébergé à Paris, qu'un utilisateur japonais souhaite visiter :

Sans CDN :

Utilisateur japonais (Tokyo) → Serveur français (Paris)

L'utilisateur japonais doit aller chercher les fichiers à Paris : toutes les requêtes traversent la planète jusqu'à Paris.

Avec CDN :

Paris → CDN mondial → redistribue à :

- Paris
- Tokyo
- New York
- ...

Donc on a avec le CDN cette configuration :

Utilisateur japonais (Tokyo) → CDN (Tokyo)

Ainsi, l'utilisateur japonais récupère les fichiers depuis un serveur CDN situé à Tokyo (ou dans une région proche) ; le CDN lui évite un aller-retour jusqu'à Paris.

Proxy classique vs reverse proxy

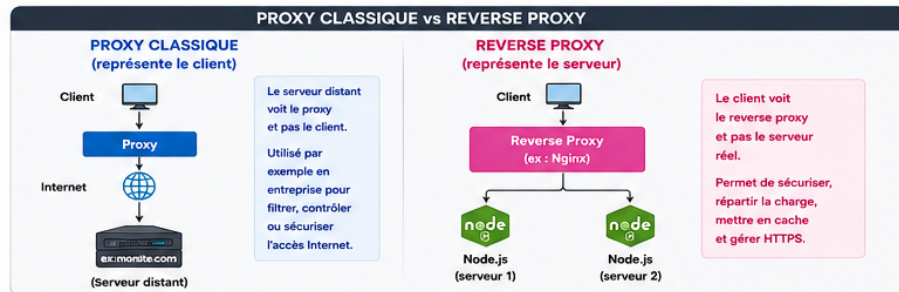


Figure 11: schéma de la différence entre proxy classique et reverse-proxy

Proxy classique Il représente le client.

Client → Proxy → monsite.com

Exemple :

- proxy d'entreprise
- VPN d'entreprise

Le serveur distant (celui qui héberge monsite.com) voit le proxy plutôt que le client ; le proxy peut servir à filtrer des sites, enregistrer le trafic, contrôler les accès, etc. pour le client (un ordinateur d'entreprise par exemple).

Reverse proxy Il représente le serveur.

Client → Reverse Proxy → Serveur réel / backend (ex : Navigateur → Nginx → Node.js)

Le client ne voit jamais le serveur backend ; il voit le reverse proxy plutôt que le backend.

Il permet de sécuriser l'accès au backend, de filtrer les requêtes, de répartir la charge entre plusieurs serveurs, de mettre en cache certaines réponses, et de gérer HTTPS.

C'est pour cela que Nginx est souvent appelé reverse proxy.

Et Internet dans tout ça ?

Internet est essentiellement un énorme réseau de routeurs reliés entre eux (les appareils qui font circuler les données entre différents réseaux).

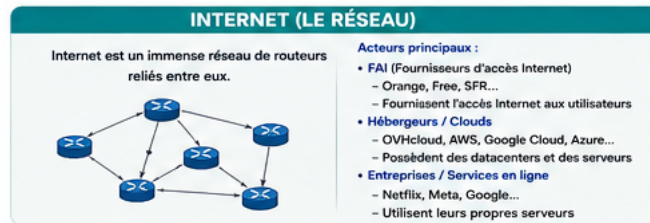


Figure 12: schéma global d'internet

Les serveurs web ne font pas tourner Internet, ils utilisent Internet.

C'est un immense réseau composé de millions d'équipements appartenant à différents acteurs:

Les FAI (Fournisseurs d'accès Internet) Exemples :

- Orange
- Free
- SFR

Ils possèdent :

- des routeurs
- des fibres
- des centres réseau

Ce sont les entreprises qui donnent l'accès à Internet aux utilisateurs.

Les Hébergeurs / Clouds Exemples :

- OVHcloud
- Amazon Web Services
- Google Cloud
- Microsoft Azure

Ils possèdent des datacenters contenant différents serveurs (serveurs web, backend, bases de données, stockage de fichiers), qui font tourner les sites web.

Entreprises Par exemple :

- Netflix
- Meta
- Google

qui possèdent aussi leurs propres serveurs.

Pour conclure :

Les 5 (parfois 6) briques d'un site moderne

Utilisateur → Serveur DNS → CDN (optionnel) → Serveur web / reverse proxy → Serveur applicatif (backend) → Serveur base de données

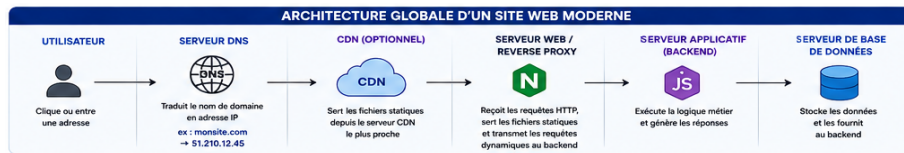


Figure 13: schéma de l'architecture globale d'un site moderne

Exemples :

- Utilisateur → Cloudflare DNS → Cloudflare CDN → Nginx → Node.js + Express → PostgreSQL

ou encore :

- Utilisateur → Route 53 (AWS) → CloudFront (AWS) → Apache → PHP + WordPress → MySQL

L'utilisateur lance une requête :

- DNS → traduit le nom de domaine en adresse IP (souvent celle du CDN)
- CDN → sert les fichiers statiques depuis un serveur proche (cache si possible)
- Serveur web / reverse proxy → reçoit les requêtes HTTP, sert les fichiers statiques et/ou les transmet au backend
- Backend (serveur applicatif) → exécute le code métier
- Base de données → stocke les données

Puis, c'est la réponse :

- La base de données renvoie les données
- Le backend construit la réponse
- Le serveur web / reverse proxy renvoie la réponse
- Le CDN peut la mettre en cache
- Le navigateur reçoit la réponse

Et pour aller plus loin :

On peut aussi ajouter l'utilisation d'HTTPS entre le client et le serveur web (souvent avec Nginx) qui repose sur des certificats SSL/TLS permettant de :

- chiffrer les communications
- garantir l'identité du site
- d'empêcher l'interception ou la modification des données pendant le transport